TestPilot AI + Gemma 4: Hackathon Plan

Oct 18, 2026 · @Void

Overview

TestPilot AI is an autonomous QA engineer: give it a web app URL and acceptance criteria, and it explores the app, runs tests in a real browser, and proves every bug with evidence. Today's job is to add Gemma 4 as its reasoning brain and make that brain visible to the judges.

One-line pitch: an AI tester that finds bugs, proves them with screenshots, and tells a real bug from a broken test.

What already works (from the running dashboard):

- Mission Control: target URL and acceptance criteria go in, a mission runs.
- Real browser testing with Stagehand and Playwright on a live journey.
- A seeded-bug sample app, so results can be measured.
- Live telemetry with discovery, analysis, execute, assertion and evidence stages.
- A generated Playwright spec, and a confirmed defect caught on screen: the form accepted mismatched passwords.

Prizes targeted: Best Use of Gemma 4 (DeepMind Google) and Best Open-Source AI Project (DigitalOcean). Submissions close today at 4:00 PM IST, and the judges score on ideation and work done, not on a finished product.

Why Gemma 4 fits both prizes

Gemma 4 is an open-weights, multimodal model reachable through the Gemini API, and TestPilot needs exactly that: a model that reads text and screenshots. Using it as the reasoning layer satisfies both prize categories at once.

Prize	What the rules ask for	How TestPilot meets it
Best Use of Gemma 4	A real Gemma call through the Gemini API, shown in the README and in code or the demo	Gemma plans tests, classifies failures and reads screenshots; the model name sits in one config variable and in the UI

Prize	What the rules ask for	How TestPilot meets it
Best Open-Source AI Project	Open-source or open-weight AI as an important part; public GitHub repo with an open-source licence; original work	Gemma is the core reasoning layer; MIT licence; public repo with a clear README

The exact Gemma 4 model ID comes from the official Gemma quickstart (mlh.link/gemma-quickstart). Copy it from there and keep it in an environment variable called `GEMMA_MODEL`.

Design choice: keep Stagehand and Playwright driving the browser, and use Gemma as a separate brain that returns short, structured JSON validated by your Zod schemas. This avoids depending on Gemma's tool-calling inside Stagehand, which is untested.

What Gemma does in the system

Gemma has four jobs, and each one returns a short JSON object with a one-sentence `reason` that the dashboard can show.

Job	Input	Output	Why it matters
Test planner	App URL and acceptance criteria	Happy-path, negative and edge scenarios with P0, P1, P2 priority	Shows the agent deciding what to test, not following a script
Visual verifier (multimodal)	Final screenshot and the expected behaviour	Pass or fail, plus what it saw	The Gemma-prize feature: it reads the image. Example: it sees a welcome message where a rejection was expected
Failure investigator	Evidence from a failed test: steps, logs, screenshot	Confirmed bug, automation error or environment failure, with confidence	Stops false bug reports, which is the hardest part of automated QA

Job	Input	Output	Why it matters
Bug report writer	Confirmed defect and its evidence	Title, severity, reproduction steps, expected vs actual	Delivers the promise of bugs proven with evidence

Flow of one mission: the planner proposes scenarios, Stagehand and Playwright run them, the visual verifier judges each result, the investigator classifies every failure, and the report writer documents confirmed bugs. A passed journey is saved as a reusable Playwright test, and a failed one is re-run to confirm it reproduces.

Making Gemma visible

Judges should see Gemma thinking at the start, during and at the end of a run. Build these in order of value:

1. **Tagged log lines.** Add `[GEMMA:PLAN]`, `[GEMMA:VERIFY]` and `[GEMMA:CLASSIFY]` to the live telemetry in their own colour, one sentence each. Example: "Generated 3 scenarios: valid registration, empty fields, mismatched passwords."
2. **"Why?" panel on the defect card.** Clicking the confirmed defect shows what Gemma expected, what it saw, why it chose "confirmed bug" over "automation error", and its confidence.
3. **Scenario plan cards.** Before the run, show the generated scenarios as cards labelled "Planned by Gemma 4" with P0, P1, P2 priority.
4. **Permanent model badge.** A header badge reading "Reasoning: Gemma 4 via Gemini API", and a small counter such as "Gemma calls: 7".
5. **Screenshot with verdict** (if capture already works). Show the failure screenshot beside Gemma's one-line verdict.

The log lines must come from the real model response, not hardcoded text. Ask Gemma for a one-sentence `reason` field in its JSON and display that, so a judge who asks "is that really Gemma?" can be shown the API call.

Features to build, in priority order

Build the top three first; they carry the idea, the Gemma requirement and the demo.

Priority	Feature	Done when
1	Gemma test planner behind the existing Zod schema	A mission produces scenarios written by Gemma, with a reason for each
2	Failure investigator	Every failed test is labelled confirmed bug, automation error or environment failure
3	Gemma visibility: tagged logs, model badge, "Why?" panel	A judge can see Gemma's reasoning on screen during the run
4	Visual verifier on the final screenshot	The mismatched-password defect is confirmed from the image
5	Rerun to confirm a bug reproduces	A failed journey is re-run and marked verified or flaky
6	Bug report writer	Report has title, severity, steps, expected vs actual, screenshot
7	Self-building test library	Passed journeys are saved as reusable Playwright tests
8	Benchmark table	Found, missed and false positives counted on the seeded bugs

If time runs short, cut in this order: report writer polish, then the visual verifier, then the test library. Never cut the planner, the investigator or the visible Gemma badge.

Honesty rules

Judges are scoring ideation and work done, so every claim has to hold up when someone asks about it.

- **Compute every dashboard number from real runs.** The 95% assertion confidence on the dashboard must either come from a calculation (for example, the share of assertions that passed on rerun) or be labelled an estimate.
- **Status slide matches reality.** An item is "built" only if it works in the demo; otherwise it stays "in progress".
- **Report results as measured.** A small honest table, such as bugs found, missed and false positives on the seeded app, beats any claim.

- **Be clear about today's work.** The prize asks for an original project. If TestPilot existed before this event, say what was built today (the Gemma integration and its visibility) and what came before.
- **Keep secrets out of the repo.** Put the Gemini API key in `.env` and add it to `.gitignore` before the first push.

Demo script (about 2 minutes)

1. **Hook.** "Friday 6 PM, a three-person startup ships a checkout update, and customers find the bug first."
2. **Start the mission.** Enter the app URL and the criterion: "Only create an account when the password and confirmation match." Point at the Gemma badge in the header.
3. **Gemma plans.** Show the scenario cards, planned by Gemma 4: valid registration, empty fields, mismatched passwords.
4. **Watch it run.** Show the live log with the `[GEMMA:...]` lines next to the browser actions.
5. **The catch.** The red defect card appears: mismatched passwords were accepted. Open the "Why?" panel and show Gemma's reasoning and the screenshot.
6. **The proof.** Show the generated Playwright spec and the rerun that confirms the bug reproduces.
7. **Close.** "Open weights, MIT licensed, and it tells a real bug from a broken test. Next: CI on every push and issue export to GitHub."

Rehearse it twice on the seeded-bug app so nothing is left to chance. Have the repo open in another tab to show the Gemma call in code if asked.

Submission checklist and risks

Submissions close today at 4:00 PM IST, so aim to submit by 3:45.

- [ ] Public GitHub repo with an open-source licence (MIT)
- [ ] README names Gemma 4 and the Gemini API, with a screenshot or GIF
- [ ] `GEMMA_MODEL` set from the official quickstart; model name visible in code and UI
- [ ] `.env` is in `.gitignore`; no API key committed
- [ ] Commit history shows work today, committed in small steps
- [ ] Small results table on the seeded bugs in the README

- [ ] Status slide updated: built, in progress, next
- [ ] Demo rehearsed twice; prototype link works
- [ ] Project submitted on the MLH event page

Risk	Fallback
Gemma returns invalid JSON	Parse and retry up to 3 times, validate with Zod, then fall back to a simple default plan
Gemma is slow or rate-limited	Cache the plan for the demo app and show real calls in the log
Visual verifier is unreliable	Keep it as a second opinion; assertions stay the source of truth
Time runs out	Ship planner, investigator and the visible Gemma badge; mark the rest as in progress